

ACTIVIDAD IX

Ingeniería Orientada a Objetos

Taller Docker — OO Clínica Web en Contenedor

CRUD de Pacientes y Citas con Django,
PostgreSQL y Podman

Universidad Surcolombiana — Ingeniería de Software

ESTUDIANTES

Jorge León Chavarro Alarcón Código: 20231211354

Juan David Rivera Chaté Código: 20231213382

DOCENTE

Eduardo Martínez Vidal

18 de abril
de 2026

Índice

1. Introducción	2
2. Objetivos	2
2.1. Objetivo general	2
2.2. Objetivos específicos	2
3. Descripción de la Aplicación	2
3.1. Dominio	2
3.2. Paradigma orientado a objetos	3
3.3. Stack tecnológico	4
4. Arquitectura de Contenedores	4
4.1. Servicio web	5
4.2. Servicio db	5
4.3. Persistencia	5
5. Archivos clave del proyecto	5
5.1. Dockerfile	6
5.2. compose.yml	6
5.3. entrypoint.sh	7
5.4. Variables de entorno	8
6. Proceso de despliegue	8
6.1. Requisitos previos	8
6.2. Pasos de despliegue	8
6.3. Credenciales del administrador	9
7. Verificación de la persistencia	9
8. Publicación en Docker Hub	9
8.1. Autenticación	9
8.2. Etiquetado	9
8.3. Publicación	10
9. Conclusiones	10
10. Referencias	10

1 Introducción

El presente informe documenta el desarrollo de la **Actividad IX** de la asignatura Ingeniería de Software Orientada a Objetos, cuyo objetivo es construir una aplicación con paradigma orientado a objetos, conectada a una base de datos relacional, empaquetada en un contenedor con persistencia de datos mediante volúmenes, y finalmente publicada en Docker Hub.

Se optó por desarrollar una aplicación web de gestión de pacientes y citas para una clínica, implementada en Python con el framework Django 6, respaldada por PostgreSQL 16 como motor de base de datos. Todo el entorno se orquesta mediante **Podman** (alternativa libre y compatible con Docker, autorizada por el docente) en lugar de Docker, aprovechando que el equipo de trabajo cuenta con sistemas operativos basados en Linux sin la necesidad de un demonio privilegiado.

2 Objetivos

2.1 Objetivo general

Desarrollar una aplicación orientada a objetos, en contenedor, con persistencia de datos mediante volúmenes y publicada en Docker Hub.

2.2 Objetivos específicos

- Implementar un CRUD relacional con al menos dos entidades siguiendo el paradigma orientado a objetos.
- Diseñar una arquitectura multi-contenedor con separación de responsabilidades entre aplicación y base de datos.
- Garantizar la persistencia de la información mediante el uso de volúmenes nombrados.
- Automatizar el arranque completo mediante `podman-compose` y un script `start.sh`.
- Publicar la imagen resultante en Docker Hub para su distribución.

3 Descripción de la Aplicación

3.1 Dominio

La aplicación modela la gestión básica de una clínica. Permite registrar pacientes con sus datos de contacto y agendar citas asociadas a cada paciente con fecha, hora y motivo. Se implementó un CRUD completo (crear, listar, editar, eliminar) para ambas entidades.

3.2 Paradigma orientado a objetos

Django implementa el patrón *Active Record* a través de su ORM: cada tabla de la base de datos se representa como una clase Python que hereda de `django.db.models.Model` y expone atributos tipados y métodos propios del dominio. A continuación se muestran las dos clases principales.

```
1 from django.db import models
2 from django.core.validators import RegexValidator
3
4
5 class Paciente(models.Model):
6     nombre = models.CharField(max_length=150)
7     cedula = models.CharField(
8         max_length=20,
9         unique=True,
10        validators=[
11            RegexValidator(
12                regex=r"^\d+$",
13                message="La cedula debe contener solo numeros.",
14            )
15        ],
16    )
17     telefono = models.CharField(
18         max_length=20,
19         validators=[
20            RegexValidator(
21                regex=r"^\+?\d+$",
22                message="El telefono debe contener solo numeros.",
23            )
24        ],
25    )
26     email = models.EmailField()
27
28     def __str__(self):
29         return f"{self.nombre} ({self.cedula})"
30
31
32 class Cita(models.Model):
33     paciente = models.ForeignKey(Paciente, on_delete=models.CASCADE)
34     fecha = models.DateTimeField()
35     motivo = models.TextField()
36
37     def __str__(self):
38         return f"{self.paciente.nombre} - {self.fecha:%Y-%m-%d %H:%M}"
```

Listing 1: clinica/models.py

Puntos clave del diseño OO:

- Cada entidad es una **clase** con atributos encapsulados y tipados.
- La clase `Paciente` incorpora **validadores** reutilizables (`RegexValidator`) que garantizan la integridad del dato a nivel de clase, no únicamente a nivel de formulario.
- La relación **1 a N** entre `Paciente` y `Cita` se expresa mediante una `ForeignKey` con política de borrado en cascada.

- El método mágico `__str__` ofrece una representación en texto útil para depuración, logs y vistas administrativas.
- El CRUD se implementa mediante *Class-Based Views* (`ListView`, `CreateView`, `UpdateView`, `DeleteView`) que heredan de clases base de Django, reutilizando lógica y promoviendo el principio de inversión de dependencias.

3.3 Stack tecnológico

Componente	Tecnología
Lenguaje	Python 3.12
Framework web	Django 6.0.4
Servidor de aplicación	Gunicorn 23
Servido de estáticos	WhiteNoise 6.9
Base de datos	PostgreSQL 16 (alpine)
Driver DB	psycopg2-binary 2.9.11
Gestión de entorno	django-environ 0.13
Contenedores	Podman 5.8 + podman-compose 1.5
Registro de imágenes	Docker Hub

Cuadro 1: Stack tecnológico del proyecto

4 Arquitectura de Contenedores

La solución se compone de **dos contenedores** orquestados por `podman-compose`, un **volumen nombrado** para la persistencia de la base de datos y una **red privada** para aislar la comunicación entre servicios.

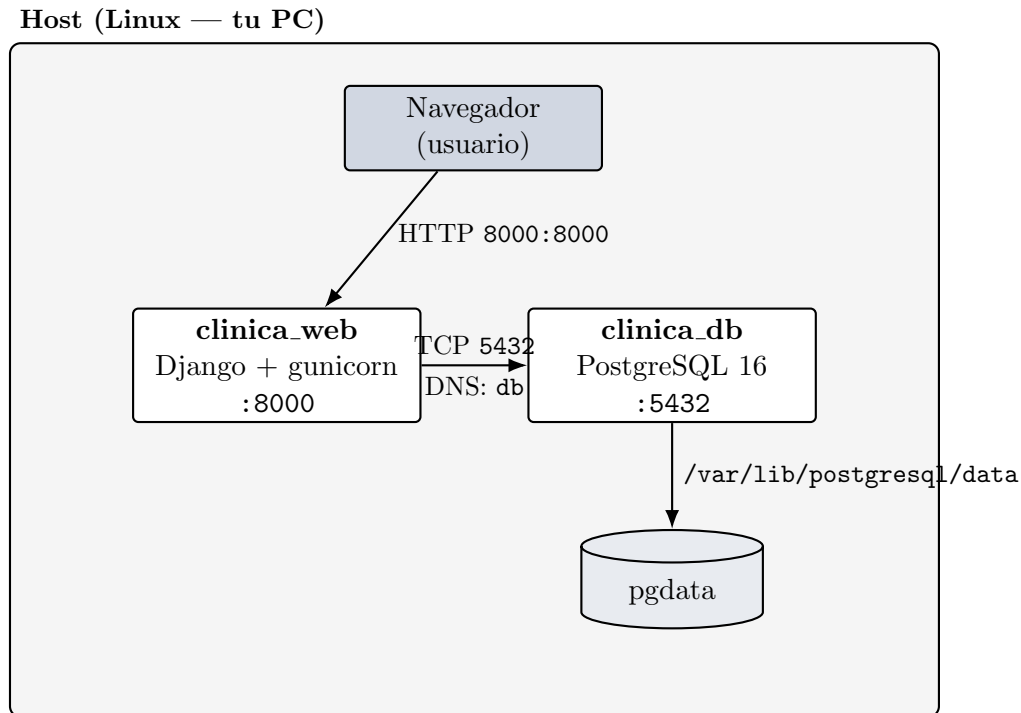


Figura 1: Arquitectura de contenedores del sistema. El navegador del usuario accede al puerto 8000 del host, que está mapeado al contenedor `clinica_web`. Éste se comunica con `clinica_db` a través de la red privada `clinica_net` usando el nombre DNS `db`. La base de datos persiste sus archivos en el volumen nombrado `pgdata`. El puerto 5432 de Postgres NO está expuesto al host.

4.1 Servicio web

Construye la imagen `clinica-ioo:latest` a partir del `Dockerfile`. Expone el puerto 8000 al host y se conecta a la base de datos a través del nombre de servicio `db`, resuelto automáticamente por la DNS interna de Podman.

4.2 Servicio db

Usa la imagen oficial `postgres:16-alpine`. No expone su puerto al host por razones de seguridad: únicamente el contenedor `web` puede comunicarse con la base a través de la red privada `clinica_net`. Implementa un *healthcheck* con `pg_isready` para que Podman conozca su estado.

4.3 Persistencia

El volumen nombrado `pgdata` se monta en `/var/lib/postgresql/data`. Esto garantiza que la información persista entre reinicios, reconstrucciones de la imagen del servicio web, y apagados del equipo. El volumen únicamente se elimina con la opción explícita `podman-compose down -v`.

5 Archivos clave del proyecto

5.1 Dockerfile

```
1 FROM python:3.12-slim AS runtime
2
3 ENV PYTHONDONTWRITEBYTECODE=1 \
4     PYTHONUNBUFFERED=1 \
5     PIP_NO_CACHE_DIR=1
6
7 WORKDIR /app
8
9 RUN apt-get update \
10    && apt-get install -y --no-install-recommends curl \
11    && rm -rf /var/lib/apt/lists/*
12
13 COPY requirements.txt .
14 RUN pip install -r requirements.txt
15
16 COPY . .
17
18 RUN chmod +x /app/entrypoint.sh \
19    && mkdir -p /app/staticfiles
20
21 EXPOSE 8000
22
23 ENTRYPOINT ["/app/entrypoint.sh"]
24 CMD ["gunicorn", "core.wsgi:application", "--bind", "0.0.0.0:8000",
     "--workers", "3", "--access-logfile", "-"]
```

Listing 2: Dockerfile

Se utiliza la imagen oficial `python:3.12-slim` como base, por su balance entre tamaño y ergonomía. El `ENTRYPOINT` se encarga de preparar la base de datos antes de ejecutar el comando principal.

5.2 compose.yml

```
1 services:
2   db:
3     image: docker.io/library/postgres:16-alpine
4     container_name: clinica_db
5     restart: unless-stopped
6     environment:
7       POSTGRES_DB: ${POSTGRES_DB}
8       POSTGRES_USER: ${POSTGRES_USER}
9       POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
10    volumes:
11      - pgdata:/var/lib/postgresql/data
12    healthcheck:
13      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
14      interval: 5s
15      timeout: 3s
16      retries: 10
17    networks:
18      - clinica_net
19
20   web:
```

```
21     build:
22         context: .
23         dockerfile: Dockerfile
24     image: clinica-ioo:latest
25     container_name: clinica_web
26     restart: unless-stopped
27     env_file:
28         - .env
29     ports:
30         - "8000:8000"
31     depends_on:
32         db:
33             condition: service_healthy
34     networks:
35         - clinica_net
36
37 volumes:
38     pgdata:
39
40 networks:
41     clinica_net:
```

Listing 3: compose.yml

5.3 entrypoint.sh

El `entrypoint` se ejecuta cada vez que arranca el contenedor web. Realiza cuatro pasos secuenciales:

1. **Espera activa a PostgreSQL:** intenta una conexión con `psycopg2` cada segundo, hasta 60 intentos.
2. **Migraciones:** ejecuta `python manage.py migrate --noinput` para aplicar el esquema.
3. **Archivos estáticos:** ejecuta `collectstatic` para que `WhiteNoise` pueda servirlos.
4. **Superusuario:** si están definidas las variables `DJANGO_SUPERUSER_USERNAME` y `DJANGO_SUPERUSER_PASSWORD`, crea o actualiza el superusuario de administración.

Finalmente ejecuta el CMD del Dockerfile (`gunicorn`) con `exec "$@"`, reemplazando el proceso del shell.

5.4 Variables de entorno

Variable	Descripción
SECRET_KEY	Clave de firma de Django.
DEBUG	Modo depuración (True/False).
ALLOWED_HOSTS	Lista separada por comas de hosts permitidos.
POSTGRES_DB	Nombre de la base.
POSTGRES_USER	Usuario de la base.
POSTGRES_PASSWORD	Contraseña de la base.
DATABASE_URL	URL completa leída por <code>django-environ</code> .
DJANGO_SUPERUSER_USERNAME	Usuario administrador creado automáticamente.
DJANGO_SUPERUSER_EMAIL	Correo del administrador.
DJANGO_SUPERUSER_PASSWORD	Contraseña del administrador.

Cuadro 2: Variables de entorno del proyecto

6 Proceso de despliegue

6.1 Requisitos previos

Sobre una distribución Linux (CachyOS / Arch en el caso de este equipo), instalar:

```
1 sudo pacman -S podman podman-compose
```

6.2 Pasos de despliegue

1. Clonar el repositorio:

```
1 git clone https://github.com/Chatesito/trabajo-ioo.git
2 cd trabajo-ioo
3
```

2. Crear el archivo `.env` a partir de la plantilla:

```
1 cp .env.example .env
2
```

3. Levantar los contenedores:

```
1 podman-compose up --build -d
2
```

4. Verificar:

```
1 podman ps
2 curl http://localhost:8000/pacientes/
3
```

Alternativamente, se incluye el script `start.sh` que automatiza todo el proceso, abre el navegador y detiene los contenedores al presionar `ENTER`.

6.3 Credenciales del administrador

El sistema crea automáticamente el siguiente superusuario en el admin de Django:

- **URL:** `http://localhost:8000/admin/`
- **Usuario:** Leon
- **Contraseña:** Leon123@

7 Verificación de la persistencia

Para demostrar que los datos no se pierden entre ejecuciones se siguió este procedimiento:

1. Se levantaron los contenedores con `podman-compose up -d`.
2. Se crearon varios pacientes y citas desde la interfaz web.
3. Se detuvieron los contenedores con `podman-compose down` (sin el flag `-v`).
4. Se volvieron a levantar con `podman-compose up -d`.
5. Se comprobó que todos los pacientes y citas seguían disponibles.

Esto es posible gracias al volumen nombrado `pgdata`, que Podman reconoce y reutiliza en todos los arranques posteriores al primero.

8 Publicación en Docker Hub

8.1 Autenticación

Desde Podman:

```
1 podman login docker.io
```

8.2 Etiquetado

```
1 podman tag clinica-foo:latest docker.io/donleonz/clinica-foo:latest
2 podman tag clinica-foo:latest docker.io/donleonz/clinica-foo:v1.0
```

8.3 Publicación

```
1 podman push docker.io/donleonz/clinica-ioo:latest
2 podman push docker.io/donleonz/clinica-ioo:v1.0
```

La imagen queda disponible en <https://hub.docker.com/r/donleonz/clinica-ioo> y cualquier persona puede descargarla y ejecutarla con:

```
1 podman pull docker.io/donleonz/clinica-ioo:latest
```

9 Conclusiones

- El uso de contenedores permite empaquetar toda la aplicación con sus dependencias de sistema, garantizando que se ejecute de la misma forma en cualquier máquina que tenga Podman o Docker instalado.
- La separación en dos contenedores (web y db) comunicados únicamente por una red privada refleja una arquitectura de producción: la base de datos nunca es accesible directamente desde el exterior.
- Los volúmenes nombrados son la única forma correcta de garantizar la persistencia de los datos en un contenedor de base de datos; el sistema de archivos interno del contenedor es efímero por diseño.
- Podman resulta una alternativa sólida a Docker en entornos Linux, por su naturaleza *rootless*, su compatibilidad con el formato OCI y la ausencia de un demonio privilegiado. Todos los comandos y archivos (`Dockerfile`, `compose.yml`) son 100 % compatibles con ambos runtimes.
- El paradigma orientado a objetos, aplicado mediante el ORM de Django, permite expresar el dominio de forma clara, con validaciones encapsuladas en las clases y relaciones explícitas entre entidades.

10 Referencias

- Documentación oficial de Django: <https://docs.djangoproject.com/en/6.0/>
- Documentación oficial de Podman: <https://docs.podman.io/>
- Repositorio del proyecto: <https://github.com/Chatesito/trabajo-ioo>
- Imagen oficial de PostgreSQL en Docker Hub: https://hub.docker.com/_/postgres
- WhiteNoise: <https://whitenoise.readthedocs.io/>